COSC428 Lab 4a - Introduction to Deep Learning for Computer Vision

Objectives

This lab introduces the general components required for applying deep learning in a computer vision context. Unlike lab 4b and the rest of the labs, there is no code to run - just examples, tips, and advice that ***should*** be helpful for your project.

The deep learning labs (and COSC428 in general) do not cover the deeper workings of Deep Learning - see COSC440 in semester 2 for more.

Table of contents

- Terminology
- Troubleshooting
- Datasets
  - Data capture
  - Annotation
  - Image augmentation
- Evaluation metrics

Troubleshooting

General troubleshooting for Lab 4b

As stated on the COSC428 Learn page, 4th year courses assume more self-directed study. Your tutors are here to help, however, we also encourage you to troubleshoot where possible.

- Avoid asking ChatGPT - it's knowledge base cutoff is early 2022, making most suggestions outdated.
- Most machine learning libraries and frameworks are well documented. A quick look through the documentation will usually point you in the right direction.
  - Pytorch
  - Detectron 2 (used for our object detection and segmentation examples)
  - YOLOv8 (not used in our labs, but becoming increasingly popular)
- If the documentation doesn't help, try Google and Stack Overflow. Chances are someone else has had the same issue.
- Post on the COSC428 Q&A forum - this can be found from the **Course Information** section on Learn.

Lab and project related examples

NOTE: Unfortunately, we do not have root permissions to install things on the lab machines.

If you need specific packages, please follow the instructions from the COSC428 Learn page to install and create a local Conda environment.

Below are the most common errors we see COSC428 students encounter while working through the deep learning lab or their projects.

Module not found: torch, detectron2 etc

```
Traceback (most recent call last):
  File "/local/428/428_Student_Projects/Lab4_test/instance_segmentation/inference_coco_dataset.py", line
    import torch
ModuleNotFoundError: No module named 'torch'
```

- **Did you activate the virtual environment?**

  ```
  source /csse/misc/course/cosc428/enviroments/detectron2/bin/activate
  ```

CUDA out of memory

```
Traceback (most recent call last):
  File "/usr/lib/python3.9/runpy.py", line 197, in _run_module_as_main
    return _run_code(code, main_globals, None,
  File "/usr/lib/python3.9/runpy.py", line 87, in _run_code
    exec(code, run_globals)
  File "/local/428_Student_Projects/Lab4_test/instance_segmentation/inference_coco_dataset.py", line 51, in <module>
    predictor = DefaultPredictor(cfg)
  File "/csse/misc/course/cosc428/enviroments/detectron2/lib/python3.9/site-packages/detectron2/engine/defaults.py", li
    self.model = build_model(self.cfg)
  File "/csse/misc/course/cosc428/enviroments/detectron2/lib/python3.9/site-packages/detectron2/modeling/meta_arch/buil
    model.to(torch.device(cfg.MODEL.DEVICE))
  File "/csse/misc/course/cosc428/enviroments/detectron2/lib/python3.9/site-packages/torch/nn/modules/module.py", line
    return self._apply(convert)
  File "/csse/misc/course/cosc428/enviroments/detectron2/lib/python3.9/site-packages/torch/nn/modules/module.py", line
    module._apply(fn)
  File "/csse/misc/course/cosc428/enviroments/detectron2/lib/python3.9/site-packages/torch/nn/modules/module.py", line
    module._apply(fn)
  File "/csse/misc/course/cosc428/enviroments/detectron2/lib/python3.9/site-packages/torch/nn/modules/module.py", line
    param_applied = fn(param)
  File "/csse/misc/course/cosc428/enviroments/detectron2/lib/python3.9/site-packages/torch/nn/modules/module.py", line
    return t.to(device, dtype if t.is_floating_point() or t.is_complex() else None, non_blocking)
RuntimeError: CUDA error: out of memory
CUDA kernel errors might be asynchronously reported at some other API call,so the stacktrace below might be incorrect.
For debugging consider passing CUDA_LAUNCH_BLOCKING=1.
```

- **Common causes & solutions:**
  - Improperly terminated Python CUDA executables.
    - Run `nvidia-smi` in your terminal, note any high memory usage Python PIDs, then use `kill PID` to kill a specific process.
  - Batch sizes are too large.
    - Edit batch size in your config.
  - Training resolution too high.
    - Reduce image size in your config.

Datasets

Data capture

For most computer vision applications, you will need to capture new training data - usually this would be a set of images containing your target object / objects; however, data capture can also include synthetic image generation, or 3D point cloud data from RGB-D cameras, 3D reconstruction, or LiDAR.

How do I capture suitable data for my application?

Before capturing any data, try to answer the following questions:

- What do you actually want to do?
- What hardware will your model be used with?
    - Camera resolution?
    - Processing requirements?
- What does the target deployment environment look like?
    - Indoor or outdoor use?
    - Potential lighting conditions?
    - Camera positioning and viewing angles?

Once you have answered these questions, you can determine what your dataset should look like. Ideally, the dataset will represent as many examples of the deployment environment as possible - e.g. don't capture only well lit, perfect lab images when you plan to operate in a dark forest.

But what about image quality?

Even if your target camera is low resolution, clearer training images will provide better features for the network to learn, and generally produce better results - where possible, you should try to capture high quality, low noise data.

The images below were captured as part of a project attempting to identify texture differences between tree trunks and branches. While both were captured at a similar resolution, the second image suffers from overexposure and a loss of detail that makes it unsuitable for training - don't delete these images though, as they can be valuable when testing edge case performance of your model.

How many images do I need?

If training a completely new model, 100's to 1000's of images are usually required. Fortunately as you will see in lab 4b, we can use **transfer learning** to reduce this, extending an existing model with under 100 annotated examples.

Annotation

In Lab 4b ***(Deep Learning for Pose and Object Recognition)***, you will annotate a playing card dataset for object detection and instance segmentation.

Typically, you will need to manually split your dataset into training, validation, and testing subsets - this split could be 75% training, 20% validation, and 5% test data (a similar split is already done for the lab examples). These must be disjoint, and only the training and validation sets need to be annotated, with samples from the test set used to validate the final model after training.

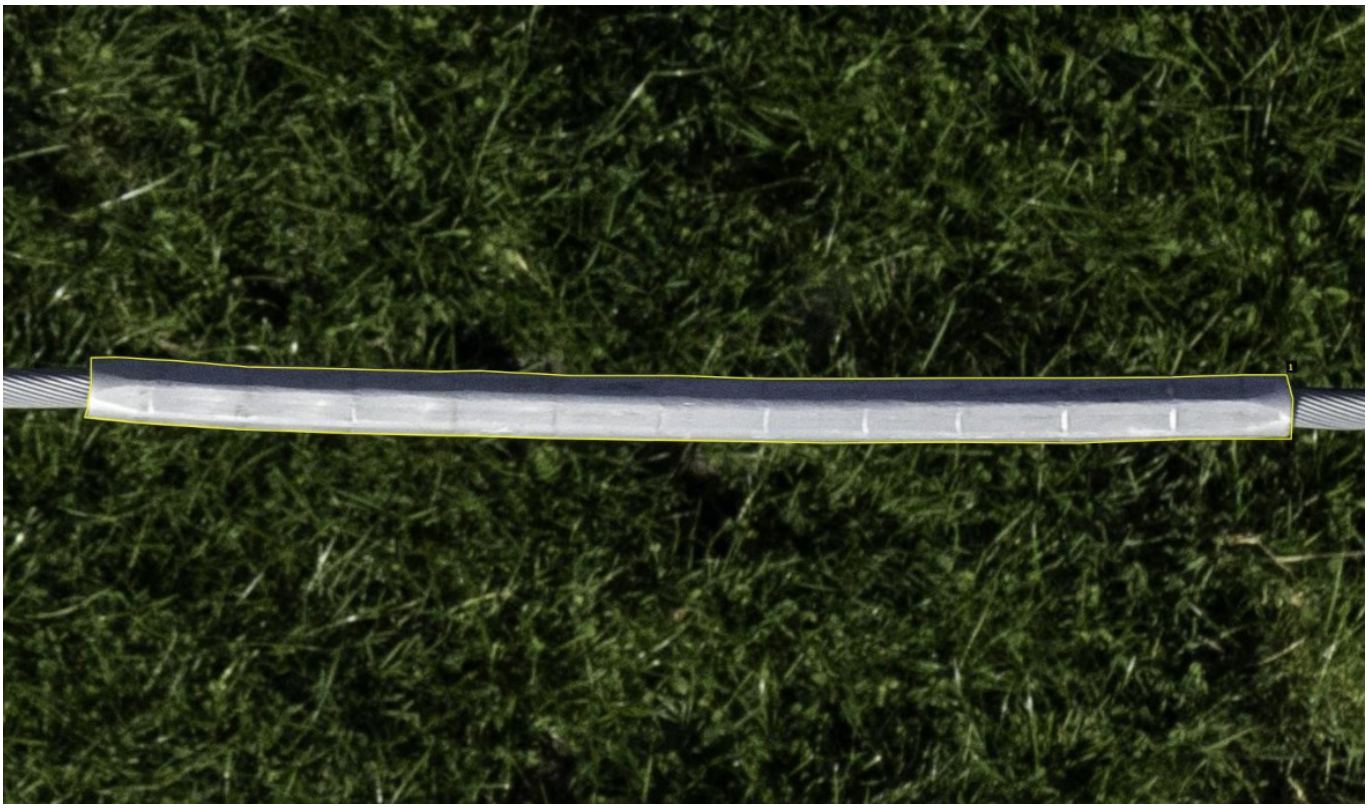
While precise annotations are generally not required for these lab examples, annotation precision can play a large part in the accuracy of your model.

PROJECT TIP: Precise data annotation can be time consuming. Start working on your annotations as soon as possible. **Not the night before project submission.**

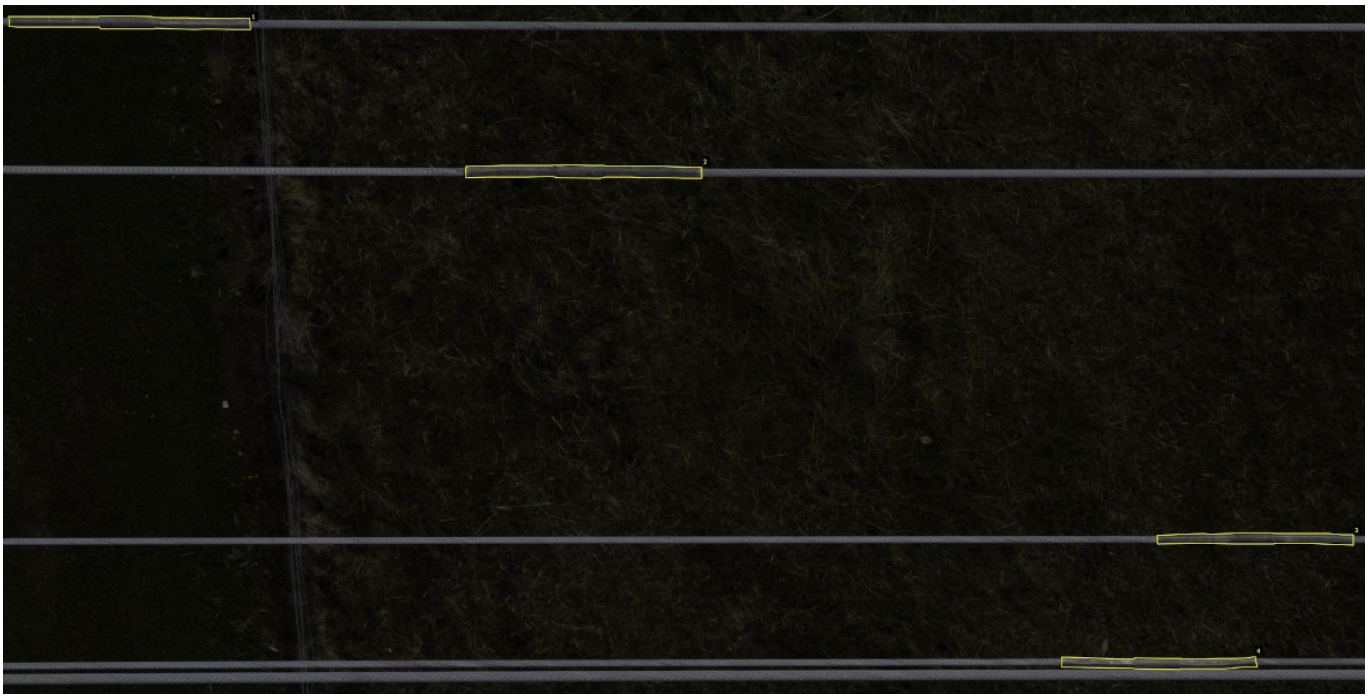
Why do we need precise annotations?

- Deep learning networks learn by capturing and encoding features from training data.
- Annotations tell the network that a specific region contains an object, and it should learn features from that region.
    - Therefore, imprecise annotations provide irrelevant, false data for the network to learn from.
    - This becomes particularly important when you are trying to detect objects against a similarly coloured or textured background.
- Less important for object detection

What makes a good annotation?

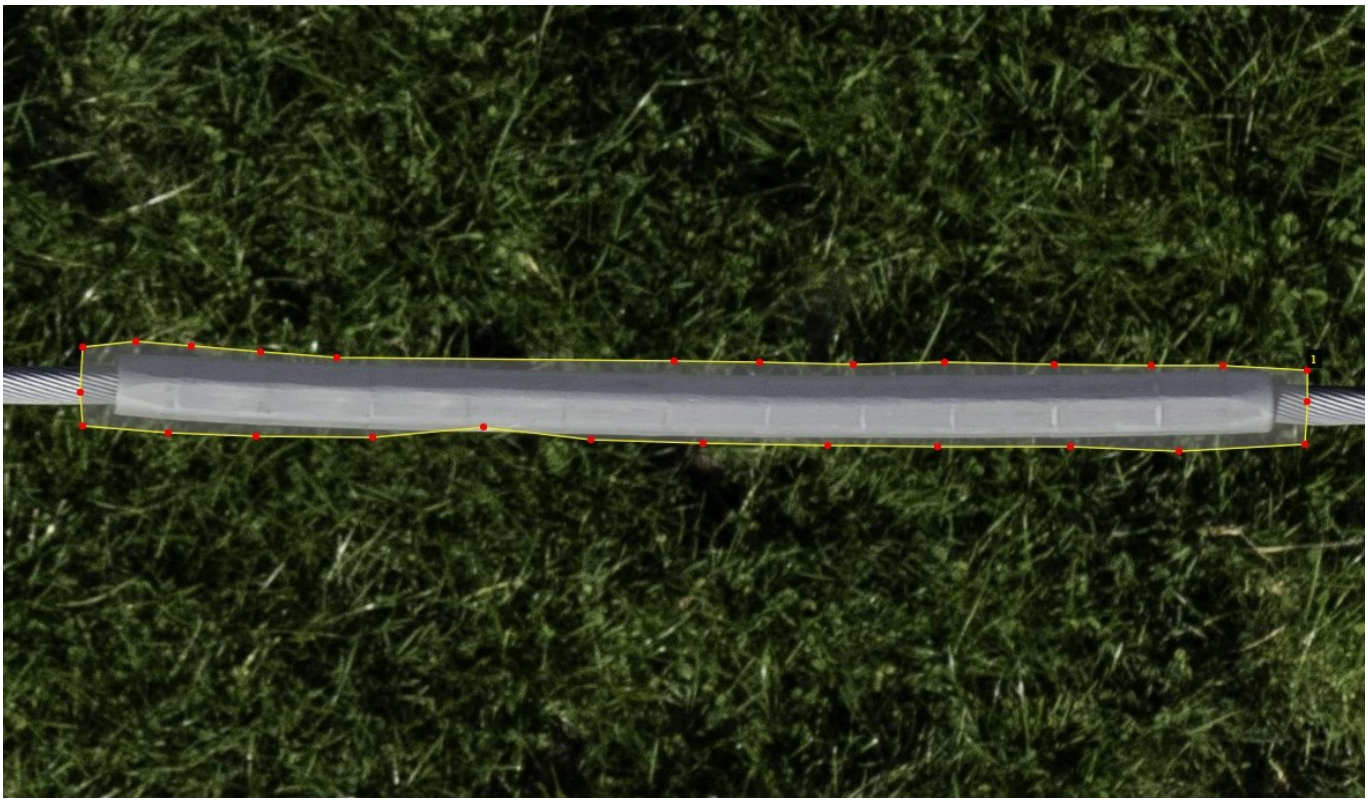


- Annotation is precise.
  - Little to no overlap with background pixels.
  - Clearly defined boundary between the object and similar adjacent regions.



- Every instance of the object is annotated.

How about bad annotations?



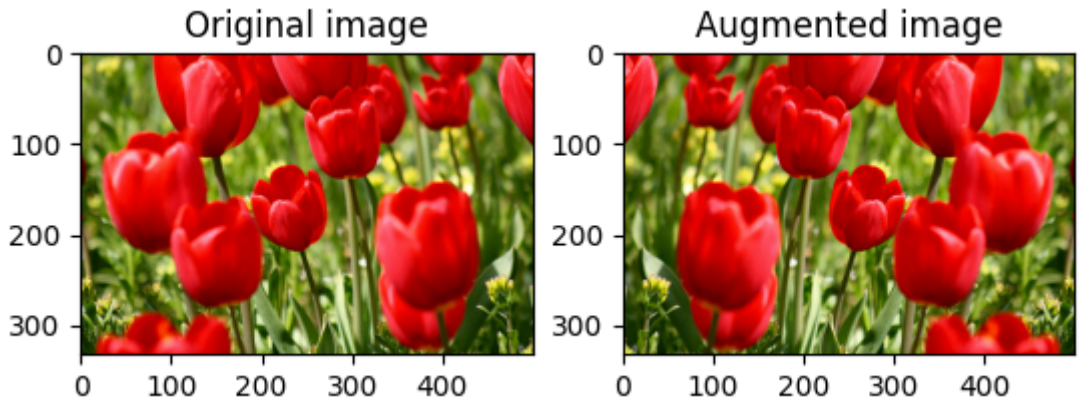
- Object is loosely annotated and contains a lot of the background pixels.
    - Highly likely that the network will learn the wrong features (the object will always appear adjacent to grass etc).

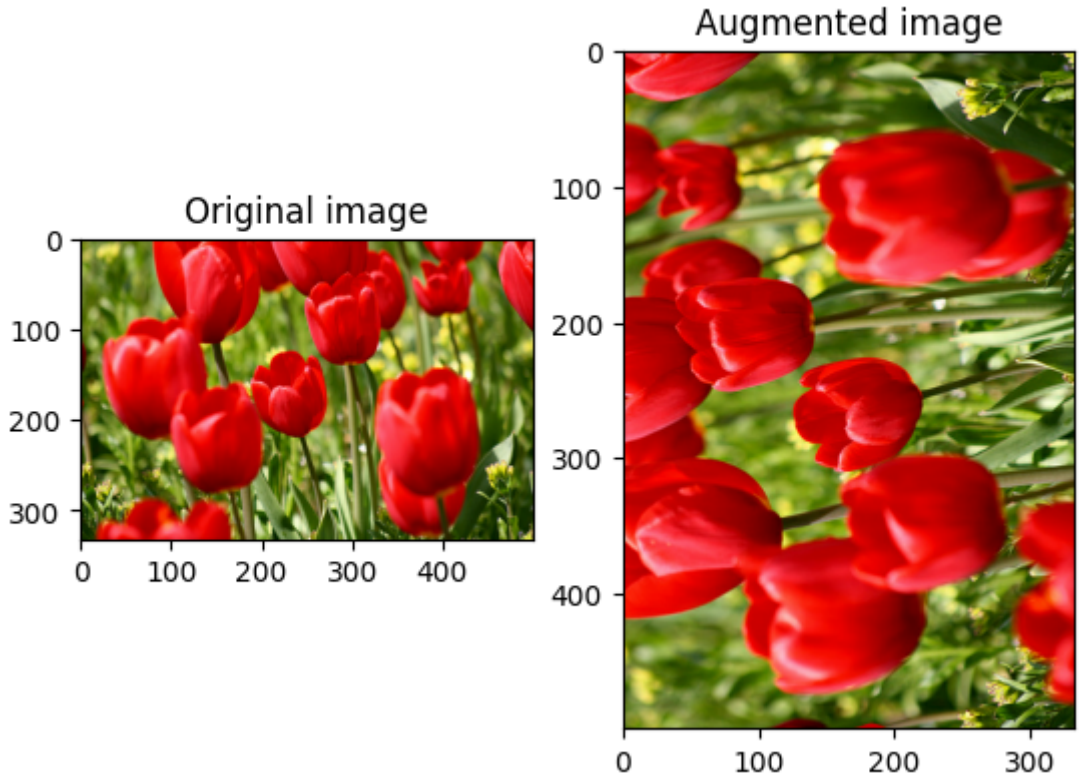
- Some instances of the object have not been annotated.
    - Unintentionally provides counterexamples when training - the network will learn to detect some objects but not others.

One final thing to consider when annotating is the similarity of other objects in the scene - in the example below, sheep, rocks, and birds all appear extremely similar from a distance, leading to false detections. Adding a *non target* class when annotating can help your network identify and ignore these objects in your output.

Image augmentation

Image augmentation is used to introduce diversity into training data, applying random but realistic transformations to data without requiring additional image capture.





Offline augmentation

Offline augmentation refers to dataset manipulation conducted manually, before training a model.

- Can be time consuming, requiring augmentations to be manually applied to data.
- Typically requires more storage, as all augmented images need to be stored as part of the overall dataset.
- Specific image augmentations can be chosen by the user, although this could introduce bias.

Online augmentation

In contrast to offline augmentation, online augmentation occurs at runtime - if enabled, the model's dataloader will apply augmentations as the data is loaded.

- Saves time
- More storage efficient than offline augmentation.
- Augmentations are typically chosen at random, making the training data more random for each training run.

Adding online augmentation to the Detectron 2 lab example

If you would like to test the effects of augmentation in the lab example, you can modify the lab training script to pass a list of augmentations to the DatasetMapper. A list of Detectron 2 augmentations is available here.

```
# Lab 4, /instance_segmentation/train.py

class Trainer(DefaultTrainer):
```

```
    @classmethod
    def build_evaluator(cls, cfg, dataset_name, output_folder=None):
        return COCOEvaluator("my_dataset_val", output_dir=cfg.OUTPUT_DIR)
    @classmethod
    def build_train_loader(cls, cfg):
        mapper = DatasetMapper(cfg, is_train=True, augmentations=
         __ADD_AUGMENTATION_LIST_HERE__  )
        return build_detection_train_loader(cfg, mapper=mapper)
```

Evaluation

Below are some evaluation metrics and measurement approaches that can help you convey model performance in your project conference paper - note that this is only a small selection, and other suitable methods may be demonstrated in lectures later in the course.

Confusion matrix

An NxN table that summarizes the number of correct and incorrect predictions made by a classification model.

Example - Cat classification

	Cat (Predicted)	**Not Cat (predicted)**
Cat (ground truth)	32 (true positive)	8 (false negative)
Not Cat (ground truth)	9 (false positive)	64 (false negative)

Accuracy

The Accuracy metric can be used to answer the following question:

How often does the model correctly predict the outcome / how does the model perform across all object classes?

$$Accuracy = \frac{correct\ predictions}{correct\ predictions + incorrect\ predictions}$$

Precision

The Precision metric can be used to answer the following question:

When the model predicted the positive class, what percentage of predictions were correct?

$$Precision = \frac{true\ positives}{true\ positives + false\ positives}$$

Recall

The Recall metric can be used to answer the following question:

When the ground truth was positive, what percentage of predictions were correctly identified as positive?

$$Recall = \frac{true\ positives}{true\ positives + false\ negatives}$$

Recall is particularly useful when dealing with class imbalanced datasets, and for these cases provides a more useful metric than accuracy.